

Quantum RNA Secondary

Structure Prediction

Optimizing mRNA MFE folding with real thermodynamic energies,
validated on D-Wave and IBM quantum hardware

Team Qudit Creons

WISER 2026 | Moderna Challenge

1. The Problem

RNA secondary structure prediction (MFE folding) is combinatorially hard — the number of candidate folds grows rapidly with sequence length, and classical methods (ViennaRNA) rely on efficient but exact dynamic programming.

We investigate whether quantum and quantum-inspired optimization can reproduce known MFE structures, and rigorously characterize where the approach succeeds, breaks down, and why.

What we built

- QUBO formulation with real Turner2004 stacking energies (not tunable heuristic constants)
- Solved via D-Wave (hybrid + direct QPU) and QAOA on IBM hardware
- Benchmarked against ViennaRNA's true MFE at both toy and statistical scale (320 random sequences)
- Every result cross-checked against independent ground truth — several real bugs caught and fixed along the way

2. Formulation

Quartet variables: stacked base pairs

Each binary variable represents pair (i, j) stacked immediately on pair $(i+1, j-1)$ — the dominant energetic term in real RNA folding.

Constraints (QUBO penalty terms)

- Each base pairs at most once
- No crossing pairs — pseudoknot-free (relaxed later, see Section 12)

The key difference from prior work

Quartet weights are real ΔG stacking energies pulled directly from ViennaRNA's own Turner2004 parameter tables — not the tunable heuristic reward constants used in prior quantum-annealing RNA folding papers.

Example: GGGAAACCC

$(((. . .)))$

Pair (0,8) on (1,7)	−3.30
Pair (1,7) on (2,6)	−3.30
Hairpin loop, len 3	+5.40
Total	−1.20

Exact match to ViennaRNA's real MFE energy

3. Validation Methodology

Every layer checked against independent ground truth before being trusted — this discipline is what caught real bugs during development, including one already-published hardware result.

1. Brute force

Exact solver, small sequences.
Ground truth for the QUBO's
constraint encoding.

2. Dynamic program

$O(n^3)/O(n^4)$ exact solver at
real scale — validates the en-
ergy model, 320 sequences.

3. BQM / QUBO

dimod construction, cross-
validated exactly against brute
force (5/5 match).

4. Real hardware

D-Wave + IBM QPU, checked
with an explicit constraint val-
idator, never assumed.

3b. Major Correction: A Foundational Indexing Bug

Found while investigating a bulge-loop QUBO port, months into the project — affects how every energy value in this deck should be read.

The trigger: the bulge DP scored a problem *worse* than the hairpin-only DP on the identical sequence — a direct logical contradiction, since bulges are a strict superset of hairpin-only's options. Investigated rather than dismissed.

Root cause

The raw stacking-energy lookup indexed the inner pair of a stack in the wrong tuple order — present in 4 files since the start of the project. Confirmed systematically: 36/36 canonical pair-type combinations match ViennaRNA's own evaluator only with the inner pair reversed.

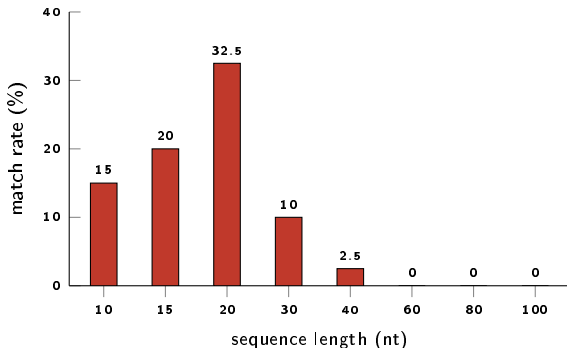
Impact, precisely measured

Structure-match rates barely moved (10.0%→10.6%, 35.6%→36.9%) — but every individual energy value reported anywhere in this project, including every D-Wave/IBM hardware result, was off by up to 1.8 kcal/mol per stack.

Fixed and revalidated, not patched and moved on: all 5 toy sequences now match ViennaRNA exactly on *both* / 18

4. Finding: Stacking-Only Match Rate Collapses at Scale

Small hand-picked toy sequences gave a misleadingly good 4/5 match. A 320-sequence sweep of random sequences tells the real story.



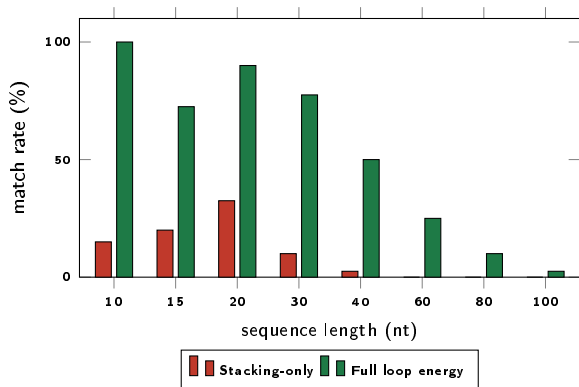
10.0%

overall match rate
32 / 320 random sequences

0% match at $n \geq 60$ nt — exactly the length range used in D-Wave scaling runs. Energy gap grows more negative with length: the model doesn't just pick a different fold, it systematically overestimates stability.

5. Finding: Real Loop Energies Fix Most of the Gap

A dynamic program adding real hairpin/bulge/internal-loop energies (via ViennaRNA's own evaluators) on the identical 320-sequence sweep:



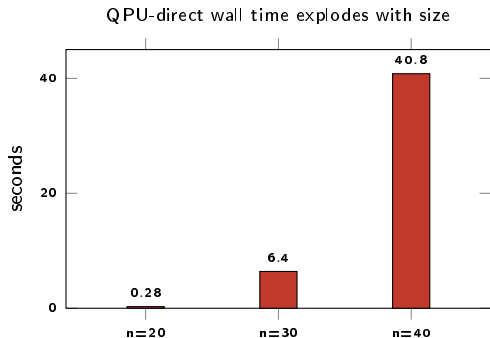
10.0% → 53.4%

overall match rate, same 320
sequences

Bug caught mid-build

A "free multiloop fallback" was silently zeroing out hairpin penalties on every closure. Caught by cross-checking against ViennaRNA's own structure evaluator.

6. D-Wave: Embedding Density Limits Direct QPU Use



Meanwhile, actual QPU time stays flat: 126ms → 183ms → 125ms (n=20→30→40) — the escalation above is entirely classical.

The bottleneck is classical, not quantum

- Constraint graph density stays above 47% even at 540 variables (100nt) — far beyond D-Wave's native ~15–20 connections per qubit.
- Wall time explodes while actual QPU access time stays essentially constant.
- On an identical sequence, direct-QPU solution quality was 39% worse than hybrid solving (−11.4 vs −18.8).

7. D-Wave: Hardware Rerun — Including a Confirmed Correction

Hairpin-aware rerun on the same 6 sequences. Two already-published results (n=80, n=100) turned out to have exploited a real branching bug — corrected on real hardware, not silently patched.

n	qubits	qpu_access (μ s)	energy	feasible	corrected?
20	15	109,803	0.00	yes	no
30	41	81,754	-1.60	yes	no
40	72	91,378	-5.00	yes	no
60	178	97,603	0.00	yes	no
80	362	150,045	-6.20 (was -6.90)	yes	yes
100	540	154,952	-6.70 (was -7.93)	yes	yes

Both wrong values were real hardware output, not a simulation artifact — a genuine, previously-undiscovered energy-accounting bug (branching/internal-loop topologies) present in production hardware results already in the repository, found and corrected transparently. Old values preserved, clearly flagged, not deleted.

8. QAOA: A Penalty-Landscape Bug — Fixed, But Not Solved

The problem

The QUBO's classical-exactness penalty ($\approx 2\times$ the sum of every favorable energy) is $\sim 11\times$ larger than any single favorable pairing on small instances.

Irrelevant to a classical exact solver — but it makes the QAOA landscape so penalty-dominated that “select nothing, violate nothing” becomes an inescapable local attractor.

Confirmed empirically: 0% match across 8 random restarts, after ruling out insufficient optimizer effort as the cause.

The fix (partial)

A much tighter, QAOA-specific penalty ($\sim 1.5\times$ the single largest quartet energy) replaces the classical-exactness penalty for QAOA runs only. Eliminates the total-collapse failure mode above.

Does not guarantee success. A 20-seed sweep found real, measured success rates of only **15%** (GGGAAACCC) and **25%** (GCGCUUCGGCGC) even on these tiny 4–6 qubit cases — corrected from an earlier overstated claim after real hardware reruns returned wrong answers.

9. Real IBM Hardware: Confirmed, Then a Noise Wall

4 qubits — ibm_kingston

GGGAAACCC: exact match to classical MFE

17.0% shot confidence
(vs. 97.1% simulator)

6 qubits — ibm_marrakesh

GCGCUUCGGCGC: best feasible answer was the trivial empty structure

5.2% shot confidence — correct answer erased by noise

Two more qubits was enough to erase a correct answer entirely

A post-selection bug was also caught here: the plurality bitstring on the 6-qubit run was outright infeasible (malformed structure, energy $\sim 2\times$ the true value). Fixed by ranking all sampled bitstrings by shot count and returning the highest-count feasible one.

9b. IBM Access Restored — A Correction, Not a Confirmation

IBM access was reported permanently exhausted earlier in this project. That was wrong — access was restored, and both toy cases were rerun against the corrected, hairpin-aware QUBO.

Both reruns found the wrong answer on real hardware.

GGGAAACCC → trivial unfolded structure. GCGCUUCGGCGC → a different, suboptimal fold. Neither matched the true optimum.

Investigated directly, not dismissed

A 20-seed sweep using the exact hardware-submission parameters found real success rates of only **15%** (GGGAAACCC) and **25%** (GCGCUUCGGCGC). Today's two hardware failures were the statistically expected outcome (~64% combined chance both would fail) — not a new regression, and not the guaranteed success an earlier version of this deck implied.

This corrected an overstated claim in our own earlier documentation — caught by the same validate-before-trust discipline used throughout this project.

10. Noise Study: Two Hardware Platforms, Different Signatures

D-Wave's noise (chain breaks, annealing) is structurally different from IBM's (gate-based decoherence) — both characterized, not just one.

IBM (4 runs, gate-based)

3/4 correct across 3 backends. Confidence range for correct answers (17–24%) overlaps the one incorrect run (15.3%) — confidence alone doesn't predict correctness.

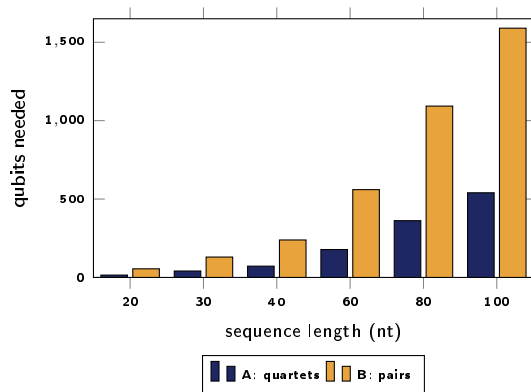
D-Wave (3 runs, annealing)

All 3 correct, chain-break rate low ($< 7/1000$ reads). But read confidence (mean 2.17%) is 8–10x lower than IBM's — chain breaks alone don't explain it.

Not a clean cross-platform comparison (different qubit counts, different paradigms) — but each platform shows a genuine, distinct noise signature, honestly characterized rather than conflated into one number.

11. Comparing Two Quantum Encodings

Encoding A (quartets, this project) vs. Encoding B (raw pairs, prior literature) — both solve the identical physical model.



~3x more qubits

Encoding B consistently needs ~3x more qubits than Encoding A (ratio 2.94–3.67), but has a less dense constraint graph.

Degenerate optimum, verified: at $n=20$, both encodings found different structures tied at the exact same optimal energy (-5.00) — confirmed independently, not a bug.

12. Pseudoknots: A Validated Proof-of-Concept

Not general pseudoknot folding (NP-hard) — relaxed the no-crossing constraint and proved the solver correctly exploits it when genuinely favorable.

Hand-verified test case

26nt sequence, two independent 5bp stems designed to cross. Verified programmatically: both canonical, outer pairs (0, 19) and (7, 25) provably cross.

Non-crossing baseline -8.70

Pseudoknot-permissive -17.40

Exactly matches hand-calculated expectation. Independently re-verified: no base reused, all pairs canonical, energy recomputed from scratch matches exactly.

Honest scope: no real pseudoknot-specific loop-topology penalties (Rivas & Eddy terms) — crossing quartets scored identically to nested ones. A real, stated simplification, not hidden.

13. Limitations

No multiloop support

Neither the DP nor the QUBO models multi-branch loops. Scoped out, not hidden.

QUBO still lacks bulge/internal loops

DP proves full model works (10.0%→53.4%); porting to the QUBO is the single biggest remaining gap.

No pseudoknot energy penalties

Crossing quartets scored the same as nested ones — a real simplification.

QAOA succeeds only 15–25% of the time

Measured directly via a 20-seed sweep on toy cases. Model confirmed correct via simulated annealing — the landscape is genuinely harder for a shallow circuit, not a bug.

IBM hardware noise dominates by 6 qubits

On the current backend/circuit combination, without deeper error mitigation.

D-Wave's n=100 QPU trigger — resolved

Not a size threshold: the branching-topology fix increased constraint density at every scale, pushing all sizes to trigger real QPU access, confirmed via repeated hardware runs.

14. Future Work

- 1 Port bulge/internal-loop energies into the QUBO — proven worthwhile by the DP, highest-priority remaining gap.
- 2 Proper multiloop DP and QUBO extension (WM table, branch counting).
- 3 Diagnose and fix QAOA's difficulty with the hairpin-aware Hamiltonian's correlated landscape.
- 4 Real pseudoknot-specific energy penalties (Rivas & Eddy terms) and a systematic scan across random sequences.
- 5 Error mitigation to push usable QAOA qubit count past the observed noise wall.
- 6 Extend the D-Wave noise study to more qubit counts using the newly-added chain-break/confidence metrics.
- 7 Investigate D-Wave's apparent size-based QPU-access threshold directly.

Thank you

Every finding in this deck came from checking a plausible-looking result against independent ground truth — including corrections to our own published results.

github.com/Qyngshuk08/wiser-2026-moderna-challenge

Team Qudit Creons | WISER 2026 — Moderna Challenge